

Requirement Analysis

FRs

FR1 - Movie Listing: The user, on selecting “**List Movies**”, shall be shown a complete list of all movies currently being screened in the cinema halls.

FR2 - Show Listing: After viewing the list of movies, the user shall be able to select a movie and view the **screen number and start time** of all shows available for that movie.

FR3- Seat Availability: After selecting a show, the user shall be able to view the seat layout along with the number of **AVAILABLE** and **BOOKED** seats.

FR4 - Seat Booking: When the customer selects one or more seats, if any selected seat is already **BOOKED**, the system shall reject the entire booking and shall not change the state of any seat.

FR5 - Pricing: The system shall calculate the total booking amount based on the type of seats selected, charging **₹150 for SILVER, ₹250 for GOLD, and ₹400 for PLATINUM** seats.

FR6 - Payment: The customer shall be able to pay by **UPI, Card or Cash**. If payment fails, the system shall **not confirm the booking**.

FR7 - Ticket Printing: On selecting this option, the user shall receive the **booking ID, screen number, seat numbers, movie name, show time, and total amount** of the corresponding show they booked.

FR8 - Booking Cancellation: On selecting this option, the seats associated with the cancelled booking shall become **AVAILABLE** again.

NFRs

NFR1 - Modularity: The system should be designed with separate classes for separate responsibilities so that each class can be developed, modified, and maintained independently.

NFR2 - Extensibility: If the cinema wants to add a new payment feature, such as NetBanking, it should be possible to implement it through a separate class without modifying the existing classes.

NFR3- Input Validation: The system shall validate user inputs such as seat numbers and menu choices, display a clear error message for invalid input, and continue without crashing.

Noun-Verb Analysis

Noun Found	Keep it as Class?	Reason
Movie	Yes	Has its own data and identity
Seat	Yes	Has number,type, price of its own
Seat Layout	No	Its a Show's seat not a thing (Just Prints)
Screen	Yes	Represents an auditorium with its own screen number and seats.
Hall Number	No	It is just an attribute/identifier of a Screen, not an independent entity.
Customer	Yes	Uniquely Identifies a customer with his name and Phone Number
Booking	Yes	Represents a customer's booking with its own booking ID, show, seats, total amount and status.
Cancellation	No	It represents an operation performed on booking not an independent entity to be made as a class
Cinema	Yes	Represents the cinema and its own theatres
Show	Yes	Represents one screening of a Movie on a Screen at a particular time.
ShowSeat	Yes	Represents the status of one physical seat for one particular Show.
Payment	Yes	Represents how the user Pays for the movie he wants to watch
UPI / Card / Cash	No	Just a method of Payment not unique identity
PriceCalculator	Yes	Has the responsibility of calculating the total price based on selected seats.
TicketPrinter	Yes	Has the responsibility of formatting and printing the ticket.
BookingService	Yes	Handles and coordinates the complete booking flow.

The Classes Needed

Class	Data Members	Member Functions	Access Modifiers	Must NOT DO
Movie	title, language, duration	getTitle(), getLanguage(), getDuration()	-attributes, +methods	Manage seats, show timings/dates, or customer information
Seat	SeatNumber, seatType	getSeatNumber(), getSeatType()	-attributes, +methods	Store or manage customer/user information
Screen	ScreenNumber, seats	getScreenNumber(), getSeats()	-attributes, +methods	Store/manage user information or movie details
Cinema	name,screens	getName(), getScreens()	-attributes, +methods	Manage individual seats, bookings, payments, or customer information
Show	movie,screen, startTime, showSeats	getMovie(),getScreen(), getStartTime(), getShowSeats();	-attributes, +methods	Manage customers or payments
ShowSeat	seat, status	getSeat(), getStatus(),bookSeat(), cancelSeat()	-attributes, +methods	Manage payments, customer information, or ticket printing
Customer	name, phone	getName(),getPhone()	-attributes, +methods	Manage Seats,payments,or ticket printing
Booking	bookingId, show, showSeats, totalAmount, status	getBookingID(), getShow(), getShowSeats(), getTotalAmount(), getStatus(), cancelBooking()	-attributes, +methods	Calculate prices, process payments, or print tickets
Payment (abstract)		pay(amount)	#members if needed, + method	Must NOT handle booking details, seat management, or ticket printing
UpiPayment		pay(amount)	+method	Must NOT manage bookings, seats, or ticket printing
CardPayment		pay(amount)	+method	Must NOT manage bookings, seats, or ticket printing
CashPayment		pay(amount)	+method	Must NOT manage bookings, seats, or ticket printing
PriceCalculator		calculateTotal(seats)	+ methods	Must NOT manage bookings, seats, or ticket printing

Relationships

Pair	Choice	Justification
Cinema – Screen	Composition	A Cinema owns its Screens. If the Cinema is bombed someday, its Screens also cease to exist as parts of that Cinema.
Screen — Seat	Composition	A Screen owns its Seats. If the Screen is removed, as part of the Screen the seats will also be removed.
Show — Movie	Aggregation	A Show borrows a movie. Cancel the show at 6 pm the movie will still play at 9PM. The show stores a pointer reference and never deletes it.
Show — Screen	Aggregation	A Show uses an existing Screen for its screening. Suppose the Show is cancelled or deleted at 6 PM; the Screen will still exist and can be used for another Show later.
Show — ShowSeat	Composition	A Show owns its ShowSeats, where each ShowSeat represents the status of a particular seat for that specific show. Suppose a Show is deleted after its screening is removed from the system; all the ShowSeats belonging to that Show will also be deleted.
Booking — Customer	Association	A Booking is associated with a Customer who makes the booking. Suppose the Booking is cancelled or deleted; the Customer will still exist in the system and can make another booking. Similarly, deleting a Customer would not necessarily mean that the Booking object must be destroyed because they have independent lifetimes.

Booking — ShowSeat	Aggregation	A Booking stores references to the ShowSeats selected by the customer. Suppose a Booking is cancelled or deleted; the ShowSeats will still exist because they belong to the Show and can become AVAILABLE again for another booking.
Booking — Payment	Association	A Booking is associated with a Payment made for that booking. Suppose the Booking is cancelled or deleted; the Payment object can still exist independently as a record of the transaction. Similarly, the Payment does not own the lifetime of the Booking.
Payment — UpiPayment	Inheritance	UpiPayment is a type of how the user will pay and hence the Payment class must be abstract which can later be inherited by UpiPayment which can be Overridden int UpiPayment to be modified into what it is actually supposed to do.
BookingService — Booking	Association	BookingService creates and manages bookings as part of the booking process, but it does not own the lifetime of a Booking. Suppose the BookingService object is destroyed; the existing Booking records can still exist independently. Similarly, a Booking can exist without the BookingService object that originally created it.

```
PS E:\system_design> cd "e:\system_design\" ; if ($?) { javac M
```

```
===== MOVIE TICKET BOOKING =====
```

1. List Movies
2. List Shows
3. Show Seats
4. Book Seat and Pay
5. Print Last Ticket
6. Cancel Last Booking
7. Exit

Enter choice: 1

Available Movies:

1. Interstellar
2. 3 Idiots
3. Inception

```
===== MOVIE TICKET BOOKING =====
```

1. List Movies
2. List Shows
3. Show Seats
4. Book Seat and Pay
5. Print Last Ticket
6. Cancel Last Booking
7. Exit

Enter choice: 2

Enter movie number: 2

Shows for 3 Idiots:

Screen 1 - 02:00 PM

```
===== MOVIE TICKET BOOKING =====
```

1. List Movies
2. List Shows
3. Show Seats
4. Book Seat and Pay
5. Print Last Ticket
6. Cancel Last Booking
7. Exit

Enter choice: 3

Available Shows:

1. Interstellar | Screen 1 | 10:00 AM
2. 3 Idiots | Screen 1 | 02:00 PM
3. Inception | Screen 2 | 06:30 PM

Enter show number: 2

Seat Layout:

Seat 1 [SILVER] : AVAILABLE

Seat 2 [SILVER] : AVAILABLE

Seat 3 [GOLD] : AVAILABLE

Seat 4 [GOLD] : AVAILABLE

Seat 5 [PLATINUM] : AVAILABLE

===== MOVIE TICKET BOOKING =====

1. List Movies

2. List Shows

3. Show Seats

4. Book Seat and Pay

5. Print Last Ticket

6. Cancel Last Booking

7. Exit

Enter choice: 4

Available Shows:

1. Interstellar | Screen 1 | 10:00 AM

2. 3 Idiots | Screen 1 | 02:00 PM

3. Inception | Screen 2 | 06:30 PM

Enter show number: 2

Seat Layout:

Seat 1 [SILVER] : AVAILABLE

Seat 2 [SILVER] : AVAILABLE

Seat 3 [GOLD] : AVAILABLE

Seat 4 [GOLD] : AVAILABLE

Seat 5 [PLATINUM] : AVAILABLE

Enter seat number to book: 5

1. UPI

2. Card

3. Cash

Choose payment method: 1

Processing UPI payment of Rs. 400.0

===== MOVIE TICKET =====

Booking ID : 1001

Movie : 3 Idiots

Screen : 1

Show Time : 02:00 PM

Seats : 5

Total : Rs. 400.0

Status : CONFIRMED

=====

===== MOVIE TICKET BOOKING =====

1. List Movies
2. List Shows
3. Show Seats
4. Book Seat and Pay
5. Print Last Ticket
6. Cancel Last Booking
7. Exit

Enter choice: 6

Booking 1001 cancelled. Seats are AVAILABLE again.

===== MOVIE TICKET BOOKING =====

1. List Movies
2. List Shows
3. Show Seats
4. Book Seat and Pay
5. Print Last Ticket
6. Cancel Last Booking
7. Exit

Enter choice: 7

PS E:\system_design> -